ЯЗЫК ZER0: РЕАЛИЗАЦИЯ, ДОПОЛНЕНИЕ, ИСПОЛЬЗОВАНИЕ

Andy Kybik

14 августа 2025 г.

Оглавление

1 Введение **5**
1.1 Цель и формат документа . . . 5
1.2 Целевая аудитория . . . 5
1.3 Обзор языка Zer0 . . . 5
1.3.1 Для кого нужен Язык Zer0? . . . 5
1.3.2 В чем сила Zer0 и где он применим, для каких практических задач создается? . . . 6
1.3.3 Примеры практических задач, для которых создается Zer0 . . . 6
1.3.4 В чем основная философия Zer0? Почему он так называется? Каким важным принципам надо следовать, какую философию держать в голове, всякий раз, программируя на Zer0? . . . 6

2 Архитектура Языка Zer0 **9**
2.0.1 Основные компоненты . . . 10
2.0.2 Описание компонентов в порядке их зависимостей . . . 10
Входная строка (inp_str) . . . 10
Алгоритм преобразований Входной строки Zer0 . . . 10
Имя Карточки (card_name) . . . 10
Значение Карточки (value, val) . . . 11
Ссылка как структура (reference, ref) . . . 12
Типы значений Карт . . . 14
Самоссылки (selflink_vals) . . . 14
Шаблонные значения (templ_vals) . . . 15
Идентификатор или Айдишник (id_vals) . . . 21
Обычное значение (usual_vals) . . . 25
Упражнения . . . 25
Алгоритм Парсера (Parser algorythm) . . . 27
Менеджер Карт (CardManager) . . . 27
Ответственности и задачи . . . 27
2.0.3 Функциональные требования . . . 27
Обработка строк . . . 27
2.0.4 Логирование . . . 27

3 Примеры использования, типичные ошибки и ‘трюки’-решения **29**

4 Доказательство Т-Полноты **31**

5 Ответы на упражнения **33**

6 Новая глава **39**

7 Глоссарий **41**

Глава 1

Введение

1.1 Цель и формат документа

Документ описывает архитектуру, функциональность и требования к языку Zer0.

Ход повествования документа построен от общего к частному, то есть, чем ближе вы от начала, тем меньше технических ответов имеете. В случае необходимости воспользуйтесь интерактивными ссылками в оглавлении.

В документе использованы ссылки, формат которых - *[терм. N]* и №страницы. Это могут быть как интерактивные одно/двусторонние ссылки, так и просто текст. В конкретном случае с терминами и их определениями это односторонняя интерактивная ссылка: из Глоссария в конце можно автоматически перейти на страницу с первым упоминанием и определением термина. В обратную сторону ссылки не переводят, так как в любом случае ведут в Глоссарий. Даже, если у вас не работают интерактивные ссылки из Глоссария, рядом есть указание страницы, на которую вела бы ссылка.

1.2 Целевая аудитория

Разработчики, которые будут реализовывать интерпретацию Zer0, расширять его как Язык или просто использовать.

1.3 Обзор языка Zer0

Zer0 — это минималистичный язык для обработки строк, основанный на заменах с использованием карточек. Строго говоря, Zer0 - это Язык не столько Программирования, сколько Перевода. Однако он может помочь при создании нового Языка Программирования, заточенного под определенные задачи (DSL - Domain-Specific Languages). При этом такое создание может одновременно сопровождаться и проектированием, и реализацией - в общем смысле в гибком режиме ‘эксперимент’.

1.3.1 Для кого нужен Язык Zer0?

Язык Зеро предназначен для разработчиков, которые имеют дело с обработкой текстовых данных и нуждаются в гибком инструменте для выполнения сложных замен и преобразований. Он будет полезен:

- **Разработчикам, работающим с текстовыми данными:** Например, для обработки логов, трансформации данных, парсинга текстовых форматов и пр.
- **Создателям DSL:** Зеро может быть использован как основа для создания специализированных языков, где требуется гибкость в определении правил замены.

- **Исследователям и аналитикам:** Для автоматизации обработки текстовых данных в исследовательских задачах.

1.3.2 В чем сила Zer0 и где он применим, для каких практических задач создается?

Сила языка Зеро заключается в его минимализме и гибкости. Он позволяет пользователю полностью контролировать процесс замены (преобразования текста на разных этапах), не ограничиваясь предопределенными командами. В Zer0 нет ни одной встроенной и реализованной изнутри команды, откуда, собственно и само название - Zer0. Единственное, чем Zer0 оперирует изначально - это *связми* между командами, каковые команды полностью вводит и задает пользователь. Это делает Zer0 мощным инструментом для решения задач, где требуется высокая степень кастомизации и универсальности.

1.3.3 Примеры практических задач, для которых создается Zer0

1. **Трансформация текстовых данных:**
   - Преобразование данных из одного формата в другой (например, код в UML, CSV в JSON или, более сложный пример, синтаксиса DSL-ЯП в иной уже существующий для исключения этапа проектировки нового Интерпретатора, а лишь ‘подтягивания’ его ‘со стороны’).
   - Обработка логов и извлечение ключевой информации.
2. **Парсинг сложных текстовых структур:**
   - Разбор текстовых файлов с нестандартными форматами.
   - Извлечение данных из текстовых шаблонов.
3. **Автоматизация текстовых задач:**
   - Генерация текстовых отчетов на основе входных данных.
   - Автоматическая замена текстовых шаблонов в документах.
4. **Создание DSL:**
   - Zer0 может быть использован как основа для создания специализированных языков, где требуется гибкость в определении правил замены.

1.3.4 В чем основная философия Zer0? Почему он так называется? Каким важным принципам надо следовать, какую философию держать в голове, всякий раз, программируя на Zer0?

Философия Zer0 заключается в **минимализме** и **максимальной гибкости**. Отчасти он похож на язык регулярных выражений, только более расширенный и универсальный. Например, это выражается в том, что язык regex не Полон по Тьюрингу, а Zer0 - да.

Также Язык (не столько Программирования, сколько Перевода) Zer0 от части похож ввиду своих принципов **минимализма** на ЯП B**F - еще одного адепта минимализма. Однако в данном случае есть ключевое различие. ЯП B**F минималистичен скорее не по причине, а по следствию. Изначально он затевался как ‘ЯП с самым легким компилятором’, а подобная нечитаемость кода, а также число команд - лишь следствие этого условия, и ввиду того же - шуточное (насколько это можно считать шуткой) название.

Название же ”Zer0”отражает иной ключевой принцип Языка (и в этом случае причина совпадает со следствием!) — нулевое количество встроенных команд. Вместо этого пользователь сам определяет все правила замены, которые в конечном итоге могут принимать форму любых грамматик, что делает язык универсальным и адаптируемым под любые задачи.

Парадигма Zer0 FLOM:

1. **F - Flexibility - Гибкость**:

   - Представьте, что Вы решаете задачу из спектра целевых для Zer0 и ищете для этого подходящий ЯП. Процесс может затянуться не только из-за нужды изучать его правила, но и ввиду его ограничений - возможно, ваша задача решается на нем гораздо сложнее, чем вы представляли изначально, и требует более глубокого исследования выбранного Языка, а может, не решается на нем вообще.

     В случае же с использованием Zer0 Вам не надо подыскивать и учить новые ЯП с их синтаксисом и ограничениями. Всё, что нужно, — это связи между ‘карточками’. А при желании на Zer0 можно создать и свою Тьюринг-полную грамматику в нужной именно Вам форме.

   - Все это позволяет думать в первую очередь о *смысле*, отбросив в сторону все не столь важные вопросы формы.

2. **L - Linkes - Принцип Связей**:

   - В Зеро важны не сами команды, а связи между ними. Именно связи определяют, как будут интерпретироваться пользовательские команды в разных последовательностях, и как будут выполняться замены внутри Zer0 для достижения этого.

   - Это позволяет создавать сложные и гибкие правила обработки текста.

3. **O - Ontological Maximalism - Онтологический Максимализм**:

   - Всё, что вводит пользователь, считается корректным по умолчанию. Если что-то пошло не так, это не ошибка языка, а результат действий пользователя. Если выражаться лаконично-юмористически, ‘Пользователь всегда прав. А когда не прав - сам дурак.’

   - Это означает, что Вы должны быть внимательными при определении правил замен. Однако здесь работает принцип Ролей и Разделения Ответственностей: на первоначальном этапе Вам как *программисту* на Zer0 дается максимальная свобода действий, которую, вы, впрочем, можете впоследствии ограничить для *пользователя* своей личной программы перевода на Zer0.

4. **M - Minimalism - Минимализм**:

   - Зеро не требует знания и заучивания никакого перечня изначально определенных внутри него команд, условно говоря, они ‘мимикрируют‘ под любые другие, причем принцип этой ‘мимикрии‘ определяете исключительно Вы как программист на Языке Перевода Zer0.

   - Программист Программы Перевода может комбинировать внутренние сущности Zer0 - обычные значения, самоссылки, шаблонные значения и айдишники (идентификаторы) для достижения нужного результата.

Глава 2

Архитектура Языка Zer0

Принцип работы Zer0 можно сравнить с лентой и написанием на ней некоторых символов. Чем-то смахивает на Машину Тьюринга, но есть ключевые различия. Также можно провести сходство между написанием коммутаторов и конжугатов в теории Кубинга при сборке Твисти Пазлов - когда требуется заполнить ячейки-пустоты в формуле поворотами, свободно перемещаясь по строке этой формулы.

Итак, условно можно описать алгоритм работы так:

1. Есть несколько карточек с двумя сторонами (на самом деле на одной из них написано не **одно**, а **множество** значений, но ради упрощения мы положим пока что лишь одно)

2. Есть входная строка

3. Алгоритм начинает искать карточки внутри строки, по особой закономерности выбирая главное между длиной, близостью вхождения и еще некоторыми параметрами при поиске

4. Найдя карточку, алгоритм переворачивает её на обратную сторону, как в игре ‘Memory‘ или ”Поле Чудес”. В результате строка преобразуется.

5. Преобразовав строку, алгоритм должен выбрать: завершить преобразование и выдать результат или продолжить. Когда преобразования окончены, результат преобразованной строки возвращается пользователю. При этом сама строка может быть любого формата и размера. В идеале при проектировании Zer0 необходимо добиться гарантии остановки. Однако это сделало бы Zer0 Тьюринг-Неполным (невозможность решения проблемы останова доказана самим Тьюрингом), и этот вопрос может обсуждаться.

Для лучшего понимания того, как достигается заявленное, перейдем к внутреннему устройству Zer0 ‘под капотом‘ на столько, на сколько его нужно знать, чтобы использовать Zer0. Сделать это тем труднее, что при последовательном закономерном раскрытии составляющих архитектуры жертвуется последовательность логическая. Поэтому предпочтем второй подход первому.

Zer0 состоит из 5 компонентов-сущностей:

2.0.1 Основные компоненты

1. **Карточка (Card)**— *хранит одно Имя и несколько Значений (от 0 до N) при том, что нет разницы в порядке ввода карт! [терм.1]*

2. **Библиотека ('Либа', Libruary)**— *хранит Карточки, которые друг друга 'видят'. В силу того, что не имеет значимости порядок ввода карт, можем точно установить, что Библиотека - строго неупорядоченное множество. [терм.2]*

3. **Менеджер Карт (CardManager)**— *управляет Карточками, проверяет их корректность и выполняет еще некоторые 'настроечные' действия над ними (подробнее см. в соответствующем параграфе). [терм.3]*

4. **Парсер (Parser)**— *реализует Алгоритм обработки, преобразования входной строки и выполняет замены, согласно связям между Карточками. Парсер возвращает не только саму строку, но и дополнительную информацию, такую как лог-файл всех замен (процесса Перевода). [терм.4]*

5. **Логгер (Logger)**— *Логирует процесс парсинга (Перевода). [терм.5]*

2.0.2 Описание компонентов в порядке их зависимостей

Входная строка (inp_str)

Технически, как бы ни был устроен Зеро и с какими бы целями ни создавался, он - ничто без предмета изменения, преобразования - входной строки.

Входная строка Zer0 (inp_str)— *пустая или непустая строка из любых символов, допустимых в некой кодировке. Теоретически, можно рассмотреть даже процессор на своей логической схеме, реализующей алгоритм Zer0 и работающей с сигналами "1" и "0" вместо текстовых символов. Полезность и эффективность такой системы в разных задачах под вопросом, но следует её рассмотреть. Однако есть вероятность, что такая система будет даже лучше и решит проблему времязатратности Алгоритма Zer0! [терм.6]*

Алгоритм преобразований Входной строки Zer0

Алгоритм (преобразований, Перевода)— *процесс замен имен карт на их значения, а затем - значений обратно на имена, но уже в другом порядке. В некоторых случаях второй этап обратных замен может отсутствовать, если таковы соответствующие связи в картах. По факту структура связей между картами и задает Программу Перевода Zer0, являющуюся, стало быть, чисто декларативной. Также есть возможность написания программы (и не обязательно одной), сохраняющую строку неизменной. [терм.7]*

Имя Карточки (card_name)

Имя (name)— *пустая или непустая строка. Ищется и применяется в качестве ссылок при Алгоритме преобразования Входной строки (а также некоторых особых карточек Менеджером Карт). [терм.8]*

Зачем нужно:

Имя карточки используется для идентификации карточки в процессе замен. Это ключевой элемент, на который ссылаются другие карточки и их значения.

Чем особенно:

Имя карточки, равно как и значения, - это пустые или непустые строки. При этом в рамках одной библиотеки каждое имя и значение уникально, то есть одно пустое имя или одно пустое значение может содержать только одна карточка или же ни одна вообще.

Пример:

```
card1_name = ”<number>”
card1_name = ‘число’
```

Значение Карточки (value, val)

Значение (value, val)— *то, на что будет меняться строка после того, как в ней найдены имена карт. [терм.9]*

Какие бывают:

Значения карт делятся по своим задачам и механизмам их исполнения на 4 вида:

1. Простые (usual_vals)

2. Самоссыльные (самоссылки, selflink_vals)

3. Шаблонные (шаблоны, templ_vals)

4. Значения-Идентификаторы (айдишники, id_vals)

Подробнее каждый вид значения карты мы разберем в соответствующих параграфах этой Главы ниже.

Зачем нужно:

Значение карточки - это то, что обеспечивает связь этапов в упрощенном алгоритме Zer0 по преобразованию Входной строки, описанный в начале этой Главы. Для того, чтобы лучше понять роль значений карт в этом процессе, ”разглядим его поближе”.

Технический алгоритм Zer0 преобразования входной строки состоит из 3-ех пунктов:

1. **Прямые замены**— *замены по всем типам карт, кроме шаблонов: поиск совпадающего значения какой-то карты и последующая замена на её имя. По факту Прямые Замены меняют контекст, избирательно (согласно содержанию карт - по сути программы на Zer0) сокращая, увеличивая или просто изменяя этот контекст. Алгоритм может кончиться в том числе и на этом этапе, если таковы карты. [терм.10]*

2. **Шаблонные замены**— *поиск исключительно шаблонных значений и замена на шаблонное имя. По факту ”перетасовка”(первый её этап) сочетаний символов - содержания строки - в другом порядке (согласно содержанию карт - по сути программы на Zer0). Если Алгоритм дошел до этапа Шаблонных замен, миновать следующего этапа - Обратных замен - он никак не может! [терм.11]*

3. **Обратные замены**— *замена уже замененных шаблонных имен обратно на их заменяемые значения, финальная ступень ”перетасовки”содержания строки (согласно содержанию карт - по сути программы на Zer0). После исполнения этого этапа Алгоритм не может не закончиться. [терм.12]*

Таким образом значения карт выступают неким ”буфером”между начальным, заменяемым и финальным, измененным состоянием строки.

Подробнее механизм всех Замен мы разберем в параграфе Парсер данной Главы.

Пример:

```
<пример значения каждого вида
ищи в соответствующем пункте ниже>
```

Ссылка как структура (reference, ref)

Ссылка— *внутренняя сущность Zer0, к которой пользователь непосредственного доступа не имеет - он не может ввести ссылку так же, как карточку, её имя или значения. Нет непосредственного механизма ввода ссылки, но ссылки вычисляются Zer0 (а именно Менеджером Карт) на основе введенного множества Карт пользователем (то есть, введенной и сформированной неупорядоченной Библиотеки).*

Ссылка - это внутренняя конструкция Zer0 из одного имени какой-то конкретной карты и одного эволюционирующего в процессе замен значения (то есть в процессе работы Алгоритма Парсера).

При этом в разных реализациях Zer0 Ссылка как структура может как хранить все эволюционировавшие значения, так и хранить лишь одно последнее (это не должно ломать работу Алгоритма и, вообще говоря, экономнее, особенно для длинных переводов). [терм.13]

Зачем нужно:

Ссылки определяют связи между карточками. Ссылка - и есть тот самый ”буфер”между двумя состояниями строки - начального на момент Простых и Шаблонных замен, и финального после момента Обратных замен. Именно благодаря этому инструменту Zer0 абсолютно декларативен и именно потому может не иметь ни одной команды (заведомо определенной, на которых строятся все новые библиотеки любого (в стандартном понимании) ЯП после создания его базового ядра - включающего в том числе и уникальную, определяющую данный ЯП грамматику).

Пример:

В дальнейших примерах ссылкой (ref) обозначается *только имя* Ссылки как внутренней структуры Zer0 (из двух полей - собственно, имени карты, и значения). Ввиду того, что значение все время эволюционирует и может принимать любые формы в зависимости от внутристрокового контекста, значение в примерах данного параграфа опускаются - так как задача параграфа - пока лишь показать механизм ввода пользователя. Позже, когда речь пойдет об интерпретации самого Алгоритма, к ref добавится второе поле (см. Главу с лог-файлом).

Также в примерах ниже к ссылке-ref прибавлено временное поле - позиция в строке. Поле приведено для наглядности – в будущем за него будет ответственен лог-файл.

Итак, пример:

```
card1_name = ‘number’
card1_val = ‘...’

inp_str = ‘number+number’
//ref1 = {name == ‘number’, pos == 0}
//ref2 = {name == ‘number’, pos == 7}
```

Имя ссылки— *полное и точное вхождение имени другой карты, которое было впервые положено в ссылку. [терм.14]*

Зачем нужно:

Имя ссылки, после того, как оно найдено в строке, заменяется на значение карты нужного вида (в зависимости от решения Парсера в ходе выполнения Алгоритма), а далее выступает ”переменной", которая восстанавливает содержание в конце замен Алгоритма, но уже в другом порядке.

Пример:

```
//в примере выше имя обоих ссылок == ‘number’
```

Значение ссылки— *эволюционирующее значение ссылки или значение подстроки, которое последним на данный момент было заменено на данную ссылку. [терм.15]*

Зачем нужно:

Термин Значения ссылки введен справедливости ради, чтобы как-то озаглавить данный объект. По большей части оно введено, потому что без него невозможно было бы ввести следующий термин.

Пример:

```
card1_name = ‘number’
card1_vals = [‘3’]

inp_str == ‘3+3’
//пока что нет ссылок

inp_str == ”number+number”
//прямые замены - подстрок на имена ссылок
//”3> ”number”
//ref1 = {name == ”number pos == 0}
//ref2 = {name == ”number pos == 7}

inp_str == ‘number*2’
//шаблонные замены:
//”перетасовали”подстроки
//а именно - ссылки (‘number’)
//+изменили контекст

inp_str == ‘3*2’
//обратные замены - имен ссылок на самое первое значение
//”number>‘3’
```

Первичное значение ссылки— *значение подстроки, которое первым было заменено на данную ссылку. [терм.16]*

Зачем нужно:

Для финального этапа - Обратных замен, по факту восстановления значений ”переменных"-ссылок.

Пример:

```
//в примере выше первичное значение
//единственной ссылки == ‘3’
```

Вид/тип ссылки— *две ссылки имеют один и тот же тип, если их имена совпадают. [терм.17]*

Зачем нужно:

Для шаблонных замен, где надо использовать более одного типа ”переменной”. Под типом здесь имеется в виду не стандартное понимание типа переменной в обычных ЯП (такие как int, string, float, double, byte, ...), а просто разные независимые ”хранилища", никак не связан-

ные друг с другом.

Пример:

Например, без ввода типизации ссылок Zer0 был бы невозможным следующий Алгоритм Перевода:

```
card1_name = ‘int’
card1_vals = [”3”]

card2_name = ‘float’
card2_vals = [‘0.5’]

//на самом деле, нужна еще 1 карта,
//но этого мы коснемся ниже...

inp_str = ‘3+0.5+3+0.5’
//пока что нет ссылок

inp_str = ‘int+float+int+float’
//прямые замены - подстрок на имена ссылок
//ref1(type1): ”3>”int”
//ref1(type1) = {name == ‘3’, pos = [0,6]}
//ref2(type2): ‘0.5’ ->‘float’
//ref2(type2) = {name == ‘0.5’, pos = [2,8]}

inp_str = ‘(int+float)*2’
//шаблонные замены - ‘перетасовали’ подстроки,
//а именно - ссылки

inp_str = ‘(3+0.5)*2’
//обратные замены - имен ссылок каждого типа
//на первичные значение
//’int’ ->‘3’
//’float’ ->‘0.5’
```

Типы значений Карт

Самоссылки (selflink_vals)

Самоссылки (selflink_vals)— *Пустая или непустая строка. Содержит имя своей карты. Применяется для сокращения контекста. [терм.18]*

Зачем нужно:

Самоссылки используются для сокращения контекста при Алгоритме Парсера. Самоссылки полезны для обработки повторяющихся паттернов в строке (Прямые замены в таком случае принимают рекурсивный характер). Например, если есть число, состоящее из множества цифр, или слово из множества букв, или еще какая-то группа символов, проще заменить её временно на один объект более короткой длины, и оперировать уже с ним. Это дает выигрыш в производительности и лишает необходимости плодить вычисления при преобразовании Алгоритма Парсера (что будет наглядно видно в лог-файле).

Чем особенно:

Самоссылка обязана содержать имя своей карточки хотя бы один раз. ‘<number><number>’.

Пример:

```
card1_name = ‘<number>’
card1_vals = [‘1’, ‘2’, ‘<number><number>’]
//Последнее значение является самоссыльным,
//так как содержит имя карты 1.
//Вообще говоря, распределится Менеджером Карт
//в отдельное поле Карточки - Значения-самоссылки.
//Здесь этого не происходит,
//потому что мы еще не знаем тип
//первых двух значений.

//Замены Входной строки Алгоритмом Парсера:
inp_str = ‘1121’

//Прямые замены - подстрок на имена ссылок:
inp_str = ‘<number><number><number><number>’
inp_str = ‘<number><number><number>’
inp_str = ‘<number><number>’
inp_str = ‘<number>’

//Шаблонные замены и Обратные замены пропустим
//(Хотя для данной строки они вообще отсутствуют)
```

Шаблонные значения (templ_vals)

Шаблонные значения (templ_vals)— *Пустая или непустая строка. Содержит хотя бы одну ссылку хотя бы одного вида, что и в имени карты. [терм.19]*

Чем особенно:

Шаблонное значение должно содержать ссылки на другие карточки, как в самом себе (в этом шаблонном значении), так и в имени карточки, которой само принадлежит.

Зачем нужно:

Шаблонные значения ‘перемалывают‘ участок строки, сохраняя в его старой и новой версии одни и те же подстроки (значения ссылок), но меняя их порядок и контекст употребления.

Пример:

```
card1_name = ‘<number>’
card1_vals = [‘3‘]

card2_name = ‘<number>*2’
//ссылка ‘<number>’ на карту card1

card2_templ_vals = [‘<numbe>+ <number>’]
//две ссылки ‘<number>’ на карту card1
//ref1(type1) = {name == ‘<number>’, pos == 0}
//ref2(type1) = {name == ‘<number>’, pos == 9}

//в итоге у карточки 2 есть ссылки на одну и ту же карту (1)
//как в имени, так и в значении
//стало быть, карта 2
//(а именно структура из её имени и данного значения)
// == шаблон
```

```
//Замены Входной строки Алгоритмом Парсера:
inp_str = ‘3+3’

//Прямые замены - подстрок на имена ссылок:
inp_str = ‘<number>+ <number>’

//Шаблонные замены - ”перетасовали”подстроки,
//а именно - ссылки:
inp_str = ‘<number>*2’

//Обратные замены - имен ссылок каждого типа
//на первичные значение
inp_str = ‘3*2’
```

В шаблоне могут участвовать ссылки **сразу нескольких типов** - например:

```
card1_name = ‘<number>’
//имя упоминается в шаблоне карточки 3,
//стало быть, является ссылкой
card1_val = ”1”

card2_name = ‘<symbol>’
//имя упоминается в шаблоне карточки 3,
//стало быть, является ссылкой
card2_val = ”x”

card3_name = ‘(<number>+<symbol>)*2’
//ссылка ‘<number>’ на карту 1
//ref1(type1) = {name == ”<number> pos == 1}
//и ‘<symbol>‘ - на карту 2
//ref2(type2) = {name == ‘<symbol>, pos == 10’}

card3_templ_vals =
[’<number>+<number>+<symbol>+<symbol>”]
//ссылка ‘<number>’ на карту 1 и ‘<symbol>’ - на карту 2
//ref1(type1) = {name == ‘<number>’, pos == 0}
//ref2(type1) = {name == ‘<number>’, pos == 9}
//ref3(type2) = {name == ‘<symbol>’, pos == 18}
//ref4(type2) = {name == ‘<symbol>’, pos == 27}
//в карте 3 и в имени, и в значении
//есть ссылки на карточки 1 и 2.
//Стало быть, карта 3
//(а именно структура из её имени и данного значения)
// == шаблон

//Замены Входной строки Алгоритмом Парсера:
inp_str = ‘1+1+x+x’

//Прямые замены - подстрок на имена ссылок:
inp_str = ‘<number>+<number>+<symbol>+<symbol>’

//Шаблонные замены - ‘перетасовали’ подстроки,
//а именно - ссылки:
```

```
inp_str = ‘(<number>+<symbol>)*2’

//Обратные замены - имен ссылок каждого типа
//на первичные значение
inp_str = ‘(1+x)*2’
```

Другой, уже знакомый нам пример:

```
card1_name = ‘int’
//имя упоминается в шаблоне карточки 3,
//стало быть, является ссылкой
card1_vals = [‘3’]

card2_name = ‘float’
//имя упоминается в шаблоне карточки 3,
//стало быть, является ссылкой
card2_vals = [‘0.5’]
card3_name = ‘(int+float)*2’
//та самая третья карта,
//необходимая Алгоритму для Шаблонных замен
card3_templ_val = ‘int+float+int+float’
//ссылка ‘int’ на карту 1 и ‘float’ - на карту 2
//ref1(type1) = {name == ‘int’, pos == 0}
//ref2(type1) = {name == ‘int’, pos == 10}
//ref3(type2) = {name == ‘float’, pos == 4}
//ref4(type2) = {name == ‘float’, pos == 14} //в карте 3 и в имени, и в значении
//есть ссылки на карточки 1 и 2.
//Стало быть, карта 3
//(а именно структура из её имени и данного значения)
// == шаблон

//Замены Входной строки Алгоритмом Парсера:
inp_str == ‘3+0.5+3+0.5’

//Прямые замены - подстрок на имена ссылок:
inp_str == ‘int+float+int+float’

//Шаблонные замены - ‘перетасовали’ подстроки,
//а именно - ссылки:
inp_str == ‘(int+float)*2’

//Обратные замены - имен ссылок каждого типа
//на первичные значение
inp_str == ‘(3+0.5)*2’
```

Более сложный пример, демонстрирующий пользу одновременно самоссылок и шаблонов:

```
card1_name = ‘<number>’
//имя упоминается в шаблоне карточки 3,
//стало быть, является ссылкой
card1_vals = [‘1’, ’<number><number>’]
//Последнее значение распределится
```

```
//Менеджером Карт в значения-Самоссылки,
//так как содержит имя карты 1.
card2_name = ‘<number>*2’
//та самая третья карта,
//необходимая Алгоритму для Шаблонных замен
card2_templ_val = ‘<number>+<number>’
//ссылка ‘<number>’ на карту 2
//ref1(type1) = {name == ‘<number>’, pos == 0}
//ref2(type1) = {name == ‘<number>’, pos == 9}
//в карте 3 и в имени, и в значении
//есть ссылки на карточку 1.
//Стало быть, карта 3
//(а именно структура из её имени и данного значения)
// == шаблон

//Замены Входной строки Алгоритмом Парсера:
inp_str = ‘11+11’

//Прямые замены - подстрок на имена ссылок:
inp_str = ‘<number><number>+<number><number>’
//Прямые замены по Самоссылкам:
inp_str = ‘<number>+<number><number>’
inp_str = ‘<number>+<number>’

//Шаблонные замены - ‘перетасовали’ подстроки,
//а именно - ссылки, и изменили контекст:
inp_str == ‘<number>*2’

//Обратные замены - имен ссылок каждого типа
//на первичные значение
//(можно заменять последовательно,
//но это неэффективно - поэтому луче
//заменить сразу на первичные значения)
inp_str == ‘11*2’
```

Таким образом в последней Программе Перевода мы получили возможность ”перетасовывать”не просто конкретные объекты, указанные в значениях Карт, - а целые группы объектов, которых непосредственно в значениях ни одной Карты нет. А именно мы може упрощать выражений-сумм из двух любых одинаковых чисел в представлении базовой системе счисления с единичным основанием.

Точно так же можно было бы заменить

```
//Начальное состояние строки
inp_str = ‘111+111’

//Преобразования
...

//Финальное состояние строки
inp_str = ‘111*2’
```

или

```
//Начальное состояние строки
inp_str = ‘11111+11111’

//Преобразования
...

//Финальное состояние строки
inp_str = ‘11111*2’
```

Однако есть одна проблема. При шаблонных и обратных заменах, если *первичные значения* ссылок *одного типа* в шаблоне не были равны, не ясно, на какое именно из этих первичных значений менять ссылки обратно, и алгоритм сообщит о *коллизии*.

Пример:

```
card1_name = ‘<number>’
card1_vals = [‘1’, ‘2’]

card2_name = ‘<number>*2’
//ссылка ‘<number>’ на карту card1

card2_templ_vals = [‘<number>+<number>’]
//две ссылки ‘<number>’ на карту card1
//ref1(type1) = {name == ”<number> pos == 0}
//ref2(type1) = {name == ”<number> pos == 9}

//в итоге у карточки 2 есть ссылки на одну и ту же карту (1)
//как в имени, так и в значении
//стало быть, карта 2
//(а именно структура из её имени и данного значения)
// == шаблон

//Замены Входной строки Алгоритмом Парсера:
inp_str = ‘1+2’

//Прямые замены - подстрок на имена ссылок:
‘1’ ->‘<number>’
‘2’ ->‘<number>’
inp_str = ’<number>+<number>’

//Шаблонные замены - ‘перетасовали’ подстроки,
//а именно - ссылки:
inp_str = ‘<number>*2’

//Обратные замены - имен ссылок каждого типа
//на первичные значение
//Коллизия! - неизвестно, на что,
//на какое из первичных значений ссылки
//типа ‘<number>’ -на ‘1’ или ‘2’ -
//менять имя ссылки (собственно, ‘<number>’):
inp_str = ‘?*2’
```

Более сложный пример (с использованием самоссылок):

```
card1_name = ‘<number>’
//имя упоминается в шаблоне карточки 3,
//стало быть, является ссылкой
card1_vals = [‘1’, ‘<number><number>’]
//Последнее значение распределится
//Менеджером Карт в значения-Самоссылки,
//так как содержит имя карты 1.

card2_name = ‘<number>*2’
card2_templ_val = ‘<number>+<number>’
//ссылка ‘<number>’ на карту 2
//ref1(type1) = {name == ‘<number>’, pos == 0}
//ref2(type1) = {name == ‘<number>’, pos == 9}
//в карте 3 и в имени, и в значении
//есть ссылки на карточку 1.
//Стало быть, карта 3
//(а именно структура из её имени и данного значения)
// == шаблон

//Замены Входной строки Алгоритмом Парсера:
inp_str = ‘1+11’

//Прямые замены - подстрок на имена ссылок:
inp_str = ‘<number>+<number><number>’
//Прямые замены по Самоссылкам:
inp_str = ‘<number>+<number>’

//Шаблонные замены - ‘перетасовали’ подстроки,
//а именно - ссылки, и изменили контекст:
inp_str == ‘<number>*2’

//Обратные замены - имен ссылок каждого типа
//на первичные значение
//Коллизия! - неизвестно, на что,
//на какое из первичных значений ссылки
//типа ‘<number>’ - на ‘1’ или ‘11’ -
//менять имя ссылки (собственно, ‘<number>’):
inp_str == ‘?*2’
//Обратите внимание! При замене непосредственно
//и сразу на первичное значение ссылки (‘1’ или ‘11’)
//алгоритм никак не ломается - а даже наоборот,
//сокращается!
```

Помимо всего прочего, Шаблон может одновременно являться и самоссылкой!
Пример:

```
card1_name = ‘<number>’
//имя упоминается в шаблоне карточки 2,
//стало быть, является ссылкой
card1_val = ‘3’
//Данное значение позже
//перераспределится Менеджером Карт
//в особый тип Значений, просто пока что
//нам еще не известно ничего об этом типе.
```

```
card2_name = ”<number>*2”
card2_templ_val = ‘(<number>*2)’
//ссылка ‘<number>’ на карту 1
//ref1(type1) = {name == ‘<number>’, pos == 1}
//ссылка ‘<number>*2’ на саму карту 2
//ref2(type2) = {name == ‘<number>*2’, pos == 1}
//в карте 2 и в имени, и в значении
//есть ссылки на карточку 1.
//Стало быть, карта 2
//(а именно структура из её имени и данного значения)
// == шаблон,
//а также есть ссылка на саму себя,
//стало быть данное значение - самоссылка.

//Замены Входной строки Алгоритмом Парсера:
inp_str = ‘(3*2)’

//Прямые замены - подстрок на имена ссылок:
inp_str = ‘(<number>*2)’
//Прямые замены по Самоссылкам
//(в данном случае совпадают с Шаблонными
и не производятся):
inp_str = ‘<number>*2’
//Прямые замены по Шаблонам -
//‘перетасовали’ подстроки,
//а именно - ссылки, и изменили контекст:
inp_str = ‘<number>*2’

//Обратные замены - имен ссылок каждого типа
inp_str == ‘3*2’
```

Замены - как Прямые, так и Шаблонные или Обратные - будут подробнее рассмотрены в Главе Алгоритм.

Таким образом, верны три утверждения:

1. **Коллизия**— *невозможность однозначно определить кандидата среди первичных значений ссылки данного типа на обратную замену [терм.20]*

2. Шаблоны, имеющие разные *первичные значения* ссылок *одного типа*, вызывают *коллизию*.

3. Шаблоны, не вызывающие коллизию, имеют одинаковые *первичные значения* ссылок *одного типа*

Идентификатор или Айдишник (id_vals)

Идентификатор или Айдишник (id_vals)— *самоссылка, участвующая в результате парсинга обоих половин хотя бы одного шаблона. Нужда в Идентификаторах обсуловлена дополнением ими Zer0 до Тьюринг-Полноты. [терм.21]*

Зачем нужно:

Чтобы Zer0 стал Т-полным, необходимо, чтобы карточки могли не только классифицировать тип ссылок (работа шаблонов), но и менять их порядок. Но мы помним два предыдущих утверждения, что приводит к невозможности провести такую операцию - ведь любая попытка будет вести либо к разным типам ссылки, либо к коллизиям:

```
card1_name = ‘<number1>’
card1_vals = [‘1’, ‘2’]
//Опять же, данные значения позже
//перераспределятся Менеджером Карт
//в особый тип Значений, просто пока что
//нам еще не известно ничего об этом типе.

card2_name = ‘<number2>’
card2_vals = [‘один’, ‘два’]
//Тут то же самое.

//при попытке сделать шаблон
//с использованием ссылок на карты 1 и 2...

card3_name = ‘<number1>+<number2>’
card3_templ_vals = [‘<number2>+<number1>’]
//...типы ссылок <number1>и <number2>
//будут разными, а значит, и значения,
//под ними хранящиеся, нельзя будет проверить
//на 1 тип
//Например, карте 1 свойственна циферная запись,
//и только она
//А карте 2 - словесная, и только она

//в противном случае
card3_name = ‘<number>+<number>’
card3_templ_vals = [‘<number>+<number>’]
//...такая конструкция теряет смысл,
//ибо ничто не переставляет

//при подстановке же в неё
//‘1+2’
//произойдет коллизия
```

Айдишники же - и есть тот самый инструмент-дополнение до Полноты, который позволяет оперировать ссылками одного типа, все же с разными значениями.

При этом важно то, что соблюдается принцип FLOM, а именно, что пользователь может вводить любую грамматику айдишников, что разобрано на примерах ниже.

Чем особенно:

Айдишник - это самоссылка, которая перераспределяется Менеджером Карт к айдишникам при следующих условиях:

1. Самоссылка должна встречаться в результате парсинга хотя бы одного шаблонного значения.

2. Эта же самоссылка должна также встречаться в результате парсинга имени карточки, которая содержит это шаблонное значение.

Таким образом,

Имя Айдишника— *результат парсинга Алгоритмом подстроки в шаблоне (его имени и значении), на которую будет заменяться подстрока Входной строки, она же первичное значение некоторой ссылки. Технически Имя Айдишника - это второе значение ссылки, первое после первичного. [терм.22]*

Тип Айдишника— *тип ссылки, которая заменилась на данный Айдишник во время Прямых Замен [терм.23]*

Значение Айдишника— *значение ссылки, на которую надо заменить данный Айдишник во время Обратных Замен [терм.24]*

Пример:

```
card1_name = ‘<num>’
card1_vals = [‘1‘, ‘2‘]
//Эти значения автоматически перераспределятся
//Менеджером Карт в последнее, неизвестное нам
//пока что тип Значений
card1_id_vals = [‘<num_<num»‘]
//есть ссылка <num>на card1,
//стало быть, изначально это самоссылка

card2_name = ‘<num_1>+<num_2>’
card2_templ_vals = [‘<num_2>+<num_1>’]

//Применяем Алгоритм Парсера к данному шаблону,
//как будто это Входная строка, и получаем:
//и в имени, и в значении card2
//’<num_1>’ преобразуется алгоритмом Парсера
//в ‘<num_<num»’
//’<num_2>’ преобразуется алгоритмом Парсера
//в ‘<num_<num»’
//то есть ‘<num_<num»’ встречается,
//как в результате парсинга шаблона,
//так и в результате парсинга его имен
//А значит, можно смело переопределить
//данную самоссылку - в айдишник
```

При процессе Замен во Входной сроке, а именно на момент Прямых Замен, Алгоритм Парсера помнит для каждого Айдишника его значение - ”смысл” того, что под ним скрывается - то есть разницу между ’<num_1>’ и ’<num_2>’, и при том же Парсер помнит о однотипности данных ссылок (однотипность Айдишника)! Таким образом мы так можем создать программу простейшего свапа двух значений:

```
card1_name = ‘<number>’
card1_vals = [‘1’, ‘2’]
//Данные значения позже
//перераспределятся Менеджером Карт
//в особый тип Значений, его мы коснемся
//уже в следующем - и последнем параграфе
//данной Главы, среди всех в ней,
//посвященных Теории.
card_val = [‘<number<number»’]
//По умолчанию это значение распределится
//Менеджером карт в самоссылки, так как
//оно содержит имя своей карты - ‘<number>’
//Однако карта 2 содержит имя
//и шаблонное значение, которые парсятся
//Алгоритмом Парсера именно в данную
//Самоссылку, а значит, она - Айдишник!

card2_name = ‘<number1>+<number2>’
```

```
card2_templ_vals = [‘<number2>+<number1>’]
//...типы ссылок <number1>и <number2>
//теперь одинаковы, а значит, и значения,
//под ними хранящиеся, будет наконец можно
//проверить на 1 тип при разных значениях!

//Замены Входной строки Алгоритмом Парсера:
inp_str = ‘1+2’

//Прямые замены - подстрок на имена ссылок
//(а именно - айдишников):
inp_str = ‘<number>+<number>’
//Поскольку замена происходила непосредственно
//на айдишники, Алгоритм имеет это в виду
//и помнит, что под одним айдишником скрывается
//одно значение, а под другим - другое,
//однако оба - ОДНОГО вида!! (Коллизии нет!)
//(значений айдишников может быть сколько угодно!)
//В лог-файле при этом будет написано
inp_str = ‘<number1>+<number2>’

//Шаблонные замены - ‘перетасовали’ подстроки,
//а именно - ссылки, и изменили контекст.
//В лог-файле будет написано
inp_str == ‘<number2>+<number1>’

//Обратные замены - имен ссылок каждого типа
inp_str = ‘2+1’
```

Иная реализация Айдишника и наглядная демонстрация парадигмы FLOM:

```
card1_name = ‘number’
card1_vals = [‘1’, ‘2’]
//Данные значения позже
//перераспределятся Менеджером Карт
//в особый тип Значений, его мы коснемся
//уже в следующем - и последнем параграфе
//данной Главы, среди всех в ней,
//посвященных Теории.
card_val = [‘number.number’]
//Иной метод создать Айдишник:
//По умолчанию это значение распределится
//Менеджером карт в самоссылки, так как
//оно содержит имя своей карты - ‘number’
//Однако карта 2 содержит имя
//и шаблонное значение, которые парсятся
//Алгоритмом Парсера именно в данную
//Самоссылку, а значит, она - Айдишник!

card2_name = ‘number.1+number.2’
card2_templ_vals = [‘number.2+number.1’]
//...типы ссылок number.1 и number.2
//теперь одинаковы, а значит, и значения,
//под ними хранящиеся, будет наконец можно
```

```
//проверить на 1 тип при разных значениях!

//Замены Входной строки Алгоритмом Парсера:
inp_str = ‘1+2’

//Прямые замены - подстрок на имена ссылок
//(а именно - айдишников):
inp_str = ‘number+number’
//Поскольку замена происходила непосредственно
//на айдишники, Алгоритм имеет это в виду
//и помнит, что под одним айдишником скрывается
//одно значение, а под другим - другое,
//однако оба - ОДНОГО вида!! (Коллизии нет!)
//(значений айдишников может быть сколько угодно!)
//В лог-файле при этом будет написано
inp_str = ‘number.1+number.2’

//Шаблонные замены - ‘перетасовали’ подстроки,
//а именно - ссылки, и изменили контекст.
//В лог-файле будет написано
inp_str == ‘number.2+number.1’

//Обратные замены - имен ссылок каждого типа
inp_str = ‘2+1’
```

Обычное значение (usual_vals) **Обычное значение (usual_vals)**— *Пустая или непустая строка. Определяется методом исключения как не самоссылка, не шаблон и не идентификатор. Применяется в качестве значения ссылкок. [терм.25]*

Зачем нужно:

Обычные значения используются для простых замен. Это самый простой, ‘линейный’ и базовый тип замен, тот самый, который мы откладывали во всех примерах ранее. Да, он самый простой, но ввести его первым не представлялось возможным, так как для этого требовалось для начала определить все предыдущие типы Значений Карт. И вот она, та непоследовательность изложения, о которой было упомянуто в самом начале Главы.

Чем особенно:

Обычные значения не содержат ссылок ни на себя (на свое имя - иначе она бы тут же превратилась в самоссылку), ни на другие карточки и их значения.

Они определяются методом исключения: если значение не является ни самоссылкой, ни шаблонным значением, ни айдишником, то оно распределяется Менеджером Карт к обычным значениям.

Пример:

```
card1_name = ‘<number>’
card1_usual_vals = [‘1’, ‘two’]
```

Упражнения

Упражнение 1

Ответ 1, стр. 33 – нажать, чтобы перейти

Распределите значения в нужный тип (определите тип ‘?’). Что надо(/достаточно) изменить в значении, чтобы оно стало самоссыльным?:

```
card1_name = `<number>'
card1_?val = [`<number+number>']
```

Упражнение 2

Ответ 2, стр. 33 – нажать, чтобы перейти

Распределите значения в нужный тип (определите тип ‘?’):

```
card1_name = `<number>'
card1_?val = [`<number>number>']
```

Упражнение 3

Ответ 3, стр. 33 – нажать, чтобы перейти

Распределите значения в нужный тип (определите тип ‘?’):

```
card1_name = `<number*<number'
card1_?val = [`<number^2']
```

Упражнение 4

Ответ 4, стр. 34 – нажать, чтобы перейти

Распределите значения в нужный тип (определите тип ‘?’). Перечислите все места, где можно заменить ‘<number’ на ‘<number>’ всего **один** раз так, чтобы карта 2 была шаблоном (больше при этом ничего не меняется!):

```
card1_name = `<number'
card1_?val = [`1', `2']

card2_name = `<number*<number'
card2_?val = [`<number^2']
```

Упражнение 5

Ответ 5, стр. 35 – нажать, чтобы перейти

Распределите значения в нужный тип (определите тип ‘?’):

```
card1_name = `<number>'
card1_?val = [`1', `2']

card2_name = "<symbol>"card2_?val = [`a', `z']

card3_name = `<number>+<number>'
card3_?val = [`<symbol>+<symbol>']
```

Упражнение 6

Ответ 6, стр. 35 – нажать, чтобы перейти

Заполните имя карты3 так, чтобы она стала шаблоном. Что будет, если использовать в её имени имя только одной из карт 1 и 2?:

```
card1_name = `<number1>'
card1_?val = [`1']

card2_name = `<number2'
card2_?val = [`2']
```

```
card3_name = `?'
card3_?val = [`<number1>+<number2']
```

Упражнение 7

Ответ 7, стр. 36 – нажать, чтобы перейти

Распределите значения в нужный тип (определите тип ‘?’). Что нужно изменить, чтобы карта1 заимела айдишник?:

```
card1_name = `num'
card1_?vals = [`0', `3']
card1_?vals = [`num(num)']

card2_name = `num(1)+num(2)'
card2_?vals = [`num(2)+num(1)']
```

Алгоритм Парсера (Parser algorythm)

Менеджер Карт (CardManager)

Ответственности и задачи

- Проверяет карточки на корректность (несколько проверок)
- Простраивает связи между введенными карточками в Библиотеке - то есть вычисляет ссылки
- Обрабатывает и деформирует айдишники, уточняя их внутреннюю структуру
- ...

2.0.3 Функциональные требования

Обработка строк

- Замена обычных значений.
- Замена самоссылок.
- Замена шаблонных значений.
- Замена айдишников.
- Обратные замены.

2.0.4 Логирование

- Формат лога (JSON).

Глава 3

Примеры использования, типичные ошибки и ‘трюки’-решения

Глава 4

Доказательство Т-Полноты

Глава 5

Ответы на упражнения

Ответ 1

Упражнение 1, стр. 25 – нажать, чтобы перейти

Распределите значения в нужный тип (определите тип ‘?’). Что надо(/достаточно) изменить в значении, чтобы оно стало самоссылыным?:

```
card1_name = `<number>'
card1_?val = [`<number+number>']
```

Ответ:

```
? = usual

//     :
//
//
//`<number'
//
//`number>'
//
//`<number>'
//                 ,
//
```

Ответ 2

Упражнение 2, стр. 26 – нажать, чтобы перейти

Распределите значения в нужный тип (определите тип ‘?’):

```
card1_name = `<number>'
card1_?val = [`<number>number>']
```

Ответ:

```
//? = selflink

//     :
//      ,  . .
```

Ответ 3

Упражнение 3, стр. 26 – нажать, чтобы перейти

Распределите значения в нужный тип (определите тип ‘?’):

```
card1_name = `<number*<number'
card1_?val = [`<number^2']
```

Ответ:

```
//? = usual

//     :
//      ,  . .   ,
//        ,  . .
```

Ответ 4

Упражнение 4, стр. 26 – нажать, чтобы перейти

Распределите значения в нужный тип (определите тип ‘?’). Перечислите все места, где можно заменить ‘<number’ на ‘<number>’ всего **один** раз так, чтобы карта 2 была шаблоном (больше при этом ничего не меняется!):

```
card1_name = `<number'
card1_?val = [`1', `2']

card2_name = `<number*<number'
card2_?val = [`<number^2']
```

Ответ:

```
card1_name = `<number'
card1_usual_val = [`1', `2']

//? = usual

//     :
//        ,  . .
//      ,  . .

card2_name = `<number*<number'
card2_templ_val = [`<number^2']

//? = templ

//     :
//`<number' -           1
//                ,
//           2
//(                              )
// ==

//      `<number'   `<number>'   ,
//        2             ,
//                    1      2 !,
//
//        `<number'              ,
//        !
```

```
//                              ,
//
```

Ответ 5

Упражнение 5, стр. 26 – нажать, чтобы перейти

Распределите значения в нужный тип (определите тип ‘?’):

```
card1_name = `<number>'
card1_?val = [`1', `2']

card2_name = "<symbol>"card2_?val = [`a', `z']

card3_name = `<number>+<number>'
card3_?val = [`<symbol>+<symbol>']
```

Ответ:

```
card1_name = `<number>'
card1_usual_val = [`1', `2']

//? = usual

//     :
//        ,  . .
//      ,  . .

card2_name = `<symbol>'
card2_usual_val = [`a', `z']

//? = usual

//     :
//        ,  . .
//      ,  . .

card3_name = `<number>+<number>'
card3_usual_val = [`<symbol>+<symbol>']

//? = usual

//     :
//        ,  . .
//      ,  . .
//      ,         !
```

Ответ 6

Упражнение 6, стр. 26 – нажать, чтобы перейти

Заполните имя карты3 так, чтобы она стала шаблоном. Что будет, если использовать в её имени имя только одной из карт 1 и 2?:

```
card1_name = `<number1>'
card1_?val = [`1']

card2_name = `<number2'
```

```
card2_?val = [`2']

card3_name = `?'
card3_?val = [`<number1>+<number2']
```

Ответ:

```
card1_name = `<number1>'
card1_usual_val = [`1']

//? = usual

//     :
//        ,  . .
//      ,  . .

card2_name = `<number2'
card2_usual_val = [`2']

//? = usual

//     :
//        ,  . .
//      ,  . .

card3_name = `' //     , `<number2+<number1>'
card3_?val = [`<number1>+<number2']

//        3
//      ,
//`<number1>',
//
//`<number2'
//                            >=0

//                     3
//            1    2,
//    3                 ,
//                       ,
//
```

Ответ 7

Упражнение 7, стр. 27 – нажать, чтобы перейти

Распределите значения в нужный тип (определите тип ‘?’). Что нужно изменить, чтобы карта1 заимела айдишник?:

```
card1_name = `num'
card1_?vals = [`0', `3']
card1_?vals = [`num(num)']

card2_name = `num(1)+num(2)'
card2_?vals = [`num(2)+num(1)']
```

Ответ:

```
card1_name = `num'
card1_usual_vals = [`0', `3']

//? = usual (             )

//     :
//        ,  . .
//      ,  . .

card_selflink_vals = [`num(num)']
//? = selflink (             )

//     :
//      ,  . .

card2_name = `num(1)+num(2)'
card2_templ_vals = [`num(2)+num(1)']
//      1          ,
//                  `3' - >`2'
//  `0' - ->`1'

//            `2'         `3',
//  `1' -   `0'
//
//                  !
//
//
//            `num(1)'   `num(2)'
//            num(num),
//
//         `num'!
```

Глава 6

Глоссарий

Термин	Определение	Стр. (нажать, чтобы перейти)
Карточка (Card)	хранит одно Имя и несколько Значений (от 0 до N) при том, что нет разницы в порядке ввода карт!	(1, стр. 10)
Библиотека (‘Либа’, Libruary)	хранит Карточки, которые друг друга ‘видят’. В силу того, что не имеет значимости порядок ввода карт, можем точно установить, что Библиотека - строго неупорядоченное множество.	(2, стр. 10)
Менеджер Карт (CardManager)	управляет Карточками, проверяет их корректность и выполняет еще некоторые ‘настроечные’ действия над ними (подробнее см. в соответствующем параграфе).	(3, стр. 10)
Парсер (Parser)	реализует Алгоритм обработки, преобразования входной строки и выполняет замены, согласно связям между Карточками. Парсер возвращает не только саму строку, но и дополнительную информацию, такую как лог-файл всех замен (процесса Перевода).	(4, стр. 10)
Логгер (Logger)	Логирует процесс парсинга (Перевода).	(5, стр. 10)
Входная строка Zer0 (inp_str)	пустая или непустая строка из любых символов, допустимых в некой кодировке. Теоретически, можно рассмотреть даже процессор на своей логической схеме, реализующей алгоритм Zer0 и работающей с сигналами ”1” и ”0” вместо текстовых символов. Полезность и эффективность такой системы в разных задачах под вопросом, но следует её рассмотреть. Однако есть вероятность, что такая система будет даже лучше и решит проблему времязатратности Алгоритма Zer0!	(6, стр. 10)
Алгоритм (преобразований, Перевода)	процесс замен имен карт на их значения, а затем - значений обратно на имена, но уже в другом порядке. В некоторых случаях второй этап обратных замен может отсутствовать, если таковы соответствующие связи в картах. По факту структура связей между картами и задает Программу Перевода Zer0, являющуюся, стало быть, чисто декларативной. Также есть возможность написания программы (и не обязательно одной), сохраняющую строку неизменной.	(7, стр. 10)

Термин	Определение	Стр. (нажать, чтобы перейти)
Имя (name)	пустая или непустая строка. Ищется и применяется в качестве ссылок при Алгоритме преобразования Входной строки (а также некоторых особых карточек Менеджером Карт).	(8, стр. 10)
Значение (value, val)	то, на что будет меняться строка после того, как в ней найдены имена карт.	(9, стр. 11)
Прямые замены	замены по всем типам карт, кроме шаблонов: поиск совпадающего значения какой-то карты и последующая замена на её имя. По факту Прямые Замены меняют контекст, избирательно (согласно содержанию карт - по сути программы на Zer0) сокращая, увеличивая или просто изменяя этот контекст. Алгоритм может кончиться в том числе и на этом этапе, если таковы карты.	(10, стр. 11)
Шаблонные замены	поиск исключительно шаблонных значений и замена на шаблонное имя. По факту ”перетасовка”(первый её этап) сочетаний символов - содержания строки - в другом порядке (согласно содержанию карт - по сути программы на Zer0). Если Алгоритм дошел до этапа Шаблонных замен, миновать следующего этапа - Обратных замен - он никак не может!	(11, стр. 11)
Обратные замены	замена уже замененных шаблонных имен обратно на их заменяемые значения, финальная ступень ”перетасовки”содержания строки (согласно содержанию карт - по сути программы на Zer0). После исполнения этого этапа Алгоритм не может не закончиться.	(12, стр. 11)
Ссылка	внутренняя сущность Zer0, к которой пользователь непосредственного доступа не имеет - он не может ввести ссылку так же, как карточку, её имя или значения. Нет непосредственного механизма ввода ссылки, но ссылки вычисляются Zer0 (а именно Менеджером Карт) на основе введенного множества Карт пользователем (то есть, введенной и сформированной неупорядоченной Библиотеки). Ссылка - это внутренняя конструкция Zer0 из одного имени какой-то конкретной карты и одного эволюционирующего в процессе замен значения (то есть в процессе работы Алгоритма Парсера). При этом в разных реализациях Zer0 Ссылка как структура может как хранить все эволюционировавшие значения, так и хранить лишь одно последнее (это не должно ломать работу Алгоритма и, вообще говоря, экономнее, особенно для длинных переводов).	(13, стр. 12)
Имя ссылки	полное и точное вхождение имени другой карты, которое было впервые положено в ссылку.	(14, стр. 12)
Значение ссылки	эволюционирующее значение ссылки или значение подстроки, которое последним на данный момент было заменено на данную ссылку.	(15, стр. 13)

Термин	Определение	Стр. (нажать, чтобы перейти)
Первичное значение ссылки	значение подстроки, которое первым было заменено на данную ссылку.	(16, стр. 13)
Вид/тип ссылки	две ссылки имеют один и тот же тип, если их имена совпадают.	(17, стр. 13)
Самоссылки (selflink_vals)	Пустая или непустая строка. Содержит имя своей карты. Применяется для сокращения контекста.	(18, стр. 14)
Шаблонные значения (templ_vals)	Пустая или непустая строка. Содержит хотя бы одну ссылку хотя бы одного вида, что и в имени карты.	(19, стр. 15)
Коллизия	невозможность однозначно определить кандидата среди первичных значений ссылки данного типа на обратную замену	(20, стр. 21)
Идентификатор или Айдишник (id_vals)	самоссылка, участвующая в результате парсинга обоих половин хотя бы одного шаблона. Нужда в Идентификаторах обсуловлена дополнением ими Zer0 до Тьюринг-Полноты.	(21, стр. 21)
Имя Айдишника	результат парсинга Алгоритмом подстроки в шаблоне (его имени и значении), на которую будет заменяться подстрока Входной строки, она же первичное значение некоторой ссылки. Технически Имя Айдишника - это второе значение ссылки, первое после первичного.	(22, стр. 22)
Тип Айдишника	тип ссылки, которая заменилась на данный Айдишник во время Прямых Замен	(23, стр. 22)
Значение Айдишника	значение ссылки, на которую надо заменить данный Айдишник во время Обратных Замен	(24, стр. 23)
Обычное значение (usual_vals)	Пустая или непустая строка. Определяется методом исключения как не самоссылка, не шаблон и не идентификатор. Применяется в качестве значения ссылкок.	(25, стр. 25)